

Hợp nhất đa cảm biến (Multi-Sensor Fusion)

PHỤ THUỘC A/04-uoc-luong-map-va-bundle-adjustment (đồ thị nhân tử — thêm cảm biến là thêm loại nhân tử); A/06-bat-dinh-va-lan-truyen-sai-so (hiệp phương sai đúng là điều kiện bắt buộc chứ không phải điều tốt nên có); C/14-hieu-chuan-nhu-mot-nhanh-cua-slam (không ghép được hai cảm biến mà không biết chúng đặt ở đâu so với nhau). Chi tiết về IMU nằm ở ../IMU_Fusion/, về marker ở ../marker/.

TÓM TẮT

Chương này ngắn có chủ ý. Phần lớn nội dung chi tiết về từng loại cảm biến đã nằm ở các cây khác, và lặp lại nó ở đây sẽ vi phạm quy tắc một nguồn duy nhất của kho. Cái chương này đóng góp là *nguyên tắc chung* — một nguyên tắc gọn tới mức đáng ngạc nhiên — cộng một bảng tra về việc mỗi cảm biến cho gì và đòi gì. Nguyên tắc: *thêm một nguồn thông tin là thêm một loại nhân tử vào đồ thị, với hiệp phương sai đúng và cổng loại outlier riêng cho từng nguồn*. Ba vế của câu đó đều quan trọng và vế thứ ba hay bị bỏ. Nội dung gồm: nguyên tắc và ba điều kiện của nó; bảng tra sáu nhóm cảm biến với cái chúng cho, điều kiện để cái đó quan sát được, và chế độ hỏng riêng; hai trường hợp đáng nói kỹ hơn vì chúng hay bị làm sai — odometry bánh xe và GNSS; và cuối cùng là một cách nhìn về việc chọn cấu hình cảm biến như một bài toán phá gauge. Nếu chỉ nhớ một điều: *giá trị lớn nhất của một cảm biến phụ không phải độ chính xác cao hơn khi mọi thứ chạy tốt, mà là một chỗ đứng để phục hồi khi thị giác thất bại*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **nhân tử (factor)** — một số hạng của hàm mục tiêu, nối các biến mà phần dư của nó phụ thuộc vào (A/04). Thêm một cảm biến nghĩa là định nghĩa một loại nhân tử mới.
- **ràng buộc nonholonomic** — ràng buộc lên *vận tốc* chứ không lên vị trí. Với robot bánh xe không trượt ngang, vận tốc theo phương ngang thân xe bằng không — một ràng buộc miễn phí và rất hữu ích.
- **odometry bánh xe** — ước lượng chuyển động từ số vòng quay bánh xe. Cho tỉ lệ metric, rẻ, tần số cao; hỏng khi trượt.
- **de-skew** — hiệu chỉnh méo của một lượt quét LiDAR do cảm biến di chuyển trong lúc quét. Cùng bản chất với vấn đề rolling shutter ở A/02.
- **hệ toàn cục so với hệ cục bộ** — SLAM làm việc trong một hệ tùy ý do nó tự chọn (gauge); GNSS cho toạ độ trong một hệ toàn cục cố định. Ghép hai hệ đòi một phép biến đổi có bốn bậc tự do phải ước lượng.
- **cổng loại outlier (outlier gating)** — cơ chế từ chối một phép đo trước khi đưa vào bộ giải. Mỗi nguồn cần cổng riêng vì phân bố outlier của chúng khác nhau.

CÂU HỎI MỞ ĐẦU

- Q1. Thêm một cảm biến vào đồ thị là thêm một loại nhân tử — nghe như một việc cơ học. Nếu ba điều kiện phải thoả, và nói điều kiện nào hay bị bỏ nhất.
- Q2. Một GNSS rẻ tiền có sai số vài mét. Thị giác của bạn chính xác tới centimet trong ngắn hạn. Nếu lý do vẫn đáng thêm GNSS, và nói lý do đó không phải “để chính xác hơn”.
- Q3. Odometry bánh xe cho tỉ lệ metric miễn phí, chính xác, tần số cao. Vì sao nó vẫn không thay thế được IMU trong hầu hết hệ thống, và chế độ hỏng của nó khác IMU thế nào?

- Q4. Bạn dùng chung một ngưỡng Mahalanobis cho mọi loại nhân tử — cách làm gọn gàng và thống nhất. Nêu vì sao đó là một sai lầm, với một ví dụ cụ thể.
- Q5. Chọn cấu hình cảm biến cho một robot mặt đất. Nêu cách đặt câu hỏi này sao cho nó trở thành một bài toán có tiêu chí thay vì một cuộc tranh luận về sở thích.

1.1 Đặt vấn đề

Thị giác một mình có ba điểm yếu cố hữu mà không thuật toán nào chữa được, vì chúng là tính chất của phép đo chứ không của cách xử lý.

Một — không có tỉ lệ với *mono* (A/05 mục 3). Hai — hỏng khi không có vân: hành lang trắng, sương mù, bóng tối, camera bị che. Ba — tần số thấp và độ trễ cao: ba mươi khung mỗi giây với vài chục mili giây xử lý là quá chậm cho một vòng điều khiển bay.

Ba điểm yếu ấy được chữa bởi ba loại cảm biến khác nhau, và đó là lý do gần như mọi hệ thống thật đều có nhiều hơn một cảm biến.

Bài toán gốc: *ghép chúng lại thế nào cho đúng?* Câu hỏi này nghe mở, và lịch sử của lĩnh vực có nhiều câu trả lời phức tạp — các sơ đồ lọc phân tầng, các cơ chế trọng số thủ công, các máy trạng thái chuyển đổi giữa các nguồn. Phần lớn chúng đã lỗi thời.

Câu trả lời hiện đại gọn tới mức đáng ngạc nhiên, và nó là hệ quả trực tiếp của A/04: *đồ thị nhân tử không quan tâm nhân tử đến từ đâu*. Một nhân tử là một hàm phần dư cộng một hiệp phương sai. Bộ giải nhìn thấy một tập nhân tử và một tập biến; nó không biết và không cần biết nhân tử nào đến từ camera, nhân tử nào từ IMU, nhân tử nào từ GNSS.

Hệ quả là việc thêm một cảm biến trở thành một việc *cục bộ*: định nghĩa biến mới nếu cần, định nghĩa phần dư, cung cấp Jacobian và hiệp phương sai. Không có gì trong bộ giải thay đổi. Đây là lý do kiến trúc đồ thị nhân tử thắng, và nó là một lợi thế lớn so với bộ lọc, nơi thêm một cảm biến nghĩa là sửa vector trạng thái và mọi ma trận liên quan.

Nhưng — và đây là toàn bộ nội dung còn lại của chương — sự gọn gàng ấy che mất ba điều kiện. Bỏ qua chúng thì việc thêm cảm biến làm hệ thống *tệ đi*, và tệ theo cách khó chẩn đoán.

TRỰC GIÁC

Hình dung một cuộc họp mà mọi người cùng ước lượng một con số — giá của một ngôi nhà chẳng hạn.

Mỗi người đưa ra một con số kèm một mức tự tin: “khoảng ba tỉ, tôi khá chắc” hoặc “khoảng bốn tỉ, nhưng tôi chỉ đoán”. Cách hợp lý để tổng hợp là trung bình có trọng số, với trọng số theo mức tự tin.

Đó chính xác là điều đồ thị nhân tử làm, và nó giải thích cả ba điều kiện.

Điều kiện một — mức tự tin phải trung thực. Nếu một người nói “tôi rất chắc” trong khi thực ra chỉ đoán, ý kiến của người đó sẽ lấn át cả phòng. Và điều tệ nhất: cuộc họp sẽ kết luận rằng *những người khác đang sai*, chứ không phải người kia đang nói quá.

Điều kiện hai — phải biết mọi người đang nói về cùng một thứ. Nếu một người nói giá tính bằng đồng và người khác tính bằng đô la, việc trung bình hoá là vô nghĩa. Trong cảm biến, “cùng một thứ” nghĩa là cùng hệ quy chiếu và cùng mốc thời gian — tức hiệu chuẩn ngoại và đồng bộ thời gian.

Điều kiện ba — phải loại người nói nhảm, và mỗi người nói nhảm theo kiểu khác nhau. Một người thỉnh thoảng nhảm một chữ số; một người khác đôi khi nói về một ngôi nhà hoàn toàn

khác. Hai kiểu sai ấy cần hai cách phát hiện, và một quy tắc chung “loại ai lệch quá xa” sẽ bắt được kiểu thứ hai mà bỏ sót kiểu thứ nhất.

Và một điều nữa, quan trọng nhất cho thực tế. Giá trị lớn nhất của việc có nhiều người trong phòng không phải là trung bình chính xác hơn khi mọi người đều tỉnh táo. Đó là khi *một người ngất xỉu* — cuộc họp vẫn tiếp tục được.

1.2 Nguyên tắc và ba điều kiện

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất.

Nguyên tắc: *thêm một nguồn thông tin là thêm một loại nhân tử vào đồ thị*. Ba điều kiện:

A. Hiệp phương sai phải trung thực

Trọng số $\Omega = \Sigma^{-1}$ quyết định nguồn này được tin bao nhiêu so với mọi nguồn khác. Khai Σ quá nhỏ nghĩa là ra lệnh cho bộ giải tin nguồn này hơn tất cả.

Hậu quả khi sai, và nó đã được nêu ở B/08 khối *Cạm bẫy*: nếu nguồn ấy có sai lệch có hệ thống, sai lệch đó chảy vào toàn bộ nghiệm, kéo theo cả những phần vốn được ước lượng tốt. Và *không có gì cảnh báo* — phần dư của các nguồn khác tăng lên, nhưng bộ giải diễn giải điều đó là “các nguồn kia kém chính xác”.

Quy tắc thực hành: trước khi thêm một loại nhân tử, trả lời được *hiệp phương sai của nó từ đâu ra*. Ba câu trả lời hợp lệ: từ datasheet của cảm biến (đã kiểm bằng đo đặc); từ một phép đo bạn tự làm (như đo σ subpixel ở B/07 mục 4B); hoặc từ một mô hình có cơ sở vật lý. Câu trả lời *không* hợp lệ: “tôi chỉnh cho tới khi nó chạy”.

B. Hệ quy chiếu và thời gian phải đúng

Hai cảm biến đặt ở hai chỗ khác nhau trên cùng thiết bị đo hai thứ khác nhau, và quan hệ giữa chúng là phép biến đổi ngoại (C/14). Sai ngoại thì mọi phần dư mang một sai lệch có hệ thống.

Sai lệch thời gian còn nguy hiểm hơn vì nó *phụ thuộc chuyển động*: một t_d sai không gây hại gì khi thiết bị đứng yên và gây hại lớn khi nó chuyển động nhanh. Hệ quả chẩn đoán đáng nhớ: nếu hệ thống của bạn chính xác khi đi chậm và tệ khi đi nhanh, nghi ngờ t_d trước khi nghi ngờ thuật toán.

Cả hai đều nên được *thả ra làm biến* theo C/14, với cổng observability.

C. Cổng loại outlier riêng cho từng nguồn

Đây là điều kiện hay bị bỏ nhất, và là câu trả lời cho câu hỏi mở đầu thứ tư.

Mỗi nguồn có *phân bố outlier riêng*, và một cổng chung không bắt được cả hai.

Ví dụ cụ thể. Outlier thị giác là các khớp sai: chúng rải rác, độc lập, và thường có phần dư rất lớn — một cổng Mahalanobis đơn giản bắt tốt. Outlier odometry bánh xe là trượt: nó *kéo dài* (cả một đoạn đường), *tương quan* (mọi phép đo trong đoạn đó đều sai cùng chiều), và phần dư của mỗi phép đo riêng lẻ có thể *nhỏ*. Một cổng Mahalanobis trên từng phép đo sẽ không bắt được nó, vì không phép đo nào lệch đủ xa — nhưng tổng của chúng thì lệch rất xa.

Phát hiện trượt cần một cơ chế khác: so tích lũy odometry với tích lũy từ một nguồn độc lập (IMU hoặc thị giác) trên một cửa sổ, và cảnh báo khi chúng phân kỳ. Đây là một cổng ở *mức chuỗi* chứ không ở mức phép đo, và nó không thể suy ra từ cổng thị giác.

Bài học tổng quát: *cổng phải được thiết kế theo cơ chế hỏng của từng nguồn*, không theo một công thức thống nhất. Một hệ thống có n loại cảm biến cần n cơ chế phát hiện, và đó là chi phí kỹ thuật thật của việc hợp nhất.

1.3 Bảng tra: mỗi cảm biến cho gì, đòi gì

Bảng 1.1. Sáu nhóm cảm biến. Cột “cho gì” nên đọc theo ngôn ngữ gauge của A/05; cột “chế độ hỏng riêng” là cột quyết định thiết kế cổng ở mục 2C.

Cảm biến	Cho gì	Điều kiện	Chế độ hỏng riêng
IMU	Tỉ lệ metric, hướng trọng lực (roll, pitch), chuyển động tần số cao	Gia tốc phải <i>thay đổi</i> để tỉ lệ quan sát được	Trôi độ lệch; bão hoà khi va đập; nhiễu rung
Odometry bánh xe	Tỉ lệ metric, chuyển động phẳng, ràng buộc nonholonomic	Bánh không trượt; đường kính bánh đã hiệu chuẩn	Trượt — kéo dài, tương quan, phần dư từng phép đo nhỏ
Stereo	Tỉ lệ metric ngay từ khung đầu	Trong tầm hữu ích; đường cơ sở ổn định	Ngoài tầm thì thoái hoá về mono; trôi đường cơ sở theo nhiệt
LiDAR	Cấu trúc metric dày, không cần ánh sáng	Bề mặt phản xạ đủ; de-skew đúng	Hành lang dài (suy biến dọc trục); mưa, bụi, kính
GNSS	Vị trí <i>tuyệt đối</i> trong hệ toàn cục	Thấy trời; ít đa đường	Đa đường trong phố; mất tín hiệu trong nhà; sai lệch tương quan
Marker / UWB	Vị trí <i>tuyệt đối</i> trong hệ đã thiết lập	Trong tầm nhìn / tầm sóng; đã lắp đặt	Ngoài tầm; marker bị che, bắn, tháo mất

Bảng này nên đọc cùng với Bảng ở 00-map: mỗi cảm biến trong cột “cho gì” thực chất là *phá một gauge*. IMU và odometry và stereo phá gauge tỉ lệ; IMU còn ghim roll với pitch; GNSS và marker phá cả gauge tịnh tiến lẫn yaw. Nhìn theo cách đó, việc chọn cấu hình cảm biến trở thành một câu hỏi có tiêu chí — đó là câu trả lời cho câu hỏi mở đầu thứ năm.

1.4 Hai trường hợp hay bị làm sai

A. Odometry bánh xe

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và odometry bánh xe đáng một mục riêng vì nó là cảm biến *bị đánh giá thấp nhất* cho robot mặt đất.

Cái nó cho, và ba thứ này đều đáng giá. *Tỉ lệ metric* — đường kính bánh đã biết cho một thước đo, giống đường cơ sở stereo. *Tần số cao và độ trễ thấp* — thường vài trăm Hz, tốt cho vòng điều khiển. *Ràng buộc nonholonomic* — và đây là phần miễn phí ít người khai thác: với robot bánh xe không trượt ngang, vận tốc theo phương ngang thân xe *bằng không*, và với robot chạy trên sàn

phẳng, vận tốc thẳng đứng cũng bằng không. Đó là hai ràng buộc không cần cảm biến nào cả — chỉ cần biết loại robot.

Vì sao nó vẫn không thay thế IMU? Vì chế độ hỏng của hai cảm biến *bù nhau* chứ không trùng nhau. IMU trôi nhưng không bao giờ “nói dối” — nó luôn đo đúng cái nó đo, chỉ là có độ lệch. Odometry không trôi khi mọi thứ bình thường, nhưng khi bánh trượt thì nó nói dối một cách tự tin: nó báo robot đã đi ba mét trong khi robot đứng yên quay bánh.

Hai kiểu sai ấy đòi hai cách xử lý ngược nhau. Với IMU, ta *mô hình hoá* độ lệch và ước lượng nó. Với odometry, ta phải *phát hiện và loại* — không có mô hình nào cho việc trượt. Và như mục 2C đã nói, việc phát hiện ấy cần một cổng ở mức chuỗi chứ không ở mức phép đo.

Kết luận thực dụng cho robot mặt đất: *dùng cả hai*. IMU cho hướng và cho chuyển động ngắn hạn khi bánh trượt; odometry cho tỉ lệ và cho chuyển động khi IMU trôi. Chúng kiểm chéo lẫn nhau, và chính việc kiểm chéo ấy là cơ chế phát hiện trượt.

B. GNSS và hai hệ quy chiếu

Đây là câu trả lời cho câu hỏi mở đầu thứ hai.

Vấn đề đầu tiên khi thêm GNSS không phải độ chính xác mà là *hệ quy chiếu*. SLAM làm việc trong một hệ tuỳ ý mà nó tự chọn khi khởi tạo — đó chính là gauge ở A/05. GNSS cho toạ độ trong một hệ toàn cục cố định. Hai hệ liên hệ qua một phép biến đổi *chưa biết* với bốn bậc tự do: ba tịnh tiến và một yaw (roll và pitch đã bị trọng lực ghim nếu có IMU).

Phép biến đổi ấy phải được *ước lượng*, và nó là một biến trong đồ thị như mọi biến khác. Điều kiện để nó quan sát được: quỹ đạo phải đủ dài và đủ cong — một đoạn thẳng không xác định được yaw của phép căn chỉnh.

Vì sao vẫn đáng thêm GNSS dù nó kém chính xác hơn thị giác? Lý do *không* phải độ chính xác mà là *bản chất sai số*. Sai số thị giác *tích lũy* — nó nhỏ ở ngắn hạn và lớn vô hạn ở dài hạn. Sai số GNSS *bị chặn* — nó vài mét ở mọi thời điểm, hôm nay cũng như một năm sau. Hai đặc tính ấy bù nhau hoàn hảo: thị giác cho độ chính xác cục bộ, GNSS cho việc không trôi.

Đây là một ví dụ của một nguyên tắc chung đáng nhớ: *khi ghép cảm biến, hãy tìm nguồn có đặc tính sai số bù nhau, không phải nguồn chính xác nhất*. Hai cảm biến cùng trôi theo cùng kiểu không giúp được nhau; một cảm biến kém chính xác nhưng không trôi lại vô giá.

Cảnh báo về cổng, đúng theo mục 2C: sai số GNSS trong phố có *đa đường* — tín hiệu phản xạ từ toà nhà — và nó gây sai lệch *tương quan theo thời gian*, có thể hàng chục mét và kéo dài nhiều giây. Một cổng dựa trên từng phép đo sẽ không bắt được, vì hiệp phương sai mà máy thu báo cũng lạc quan trong tình huống ấy. Cần một cổng so với dự đoán từ thị giác trên một cửa sổ.

CẦN NHỚ

Giá trị lớn nhất của một cảm biến phụ không phải độ chính xác cao hơn khi mọi thứ chạy tốt, mà là một chỗ đứng để phục hồi khi nguồn chính thất bại.

Đây là điều mà các bảng so sánh độ chính xác không nắm bắt được, và nó là lý do quan trọng nhất để hợp nhất cảm biến trong sản phẩm.

Ba biểu hiện cụ thể, đã xuất hiện rải rác trong cây. *Một*: khi thị giác mất bám, odometry và IMU giữ cho hệ thống có một ước lượng thô — đủ để tiếp tục vòng điều khiển, và đủ để thu hẹp không gian tìm kiếm khi relocalize (B/12 mục 2C). *Hai*: tiên nghiệm chuyển động từ cảm biến phụ cải thiện chính front-end thị giác, theo ba đường cùng lúc (B/07 mục 5C). *Ba*: khi hai nguồn bất đồng, đó là một tín hiệu chẩn đoán — và một hệ chỉ có một nguồn thì không bao giờ có tín hiệu ấy.

Hệ quả cho việc đánh giá: đo một hệ hợp nhất bằng ATE trung bình trên một chuỗi sạch sẽ bỏ sót phần lớn giá trị của nó. Phải đo trên các chuỗi có sự cố — che camera, tắt đèn, bánh trượt — và đo tỉ lệ sống sót chứ không chỉ độ chính xác. C/18 bàn cách thiết kế phép đo đó.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
VINS-Fusion [1]	Camera + IMU + GNSS tùy chọn; căn chỉnh hệ cục bộ với hệ toàn cục	Mục 4B trong một hệ thật; mã ngắn dễ đọc
GTSAM [2]	Thư viện nhân tử cho nhiều loại cảm biến; dễ thêm loại mới	Hiện thân của nguyên tắc ở mục 2 — thêm cảm biến là viết một lớp nhân tử
Kimera [3]	Camera + IMU + ngữ nghĩa trong một đồ thị	Cho thấy cả thông tin ngữ nghĩa cũng chỉ là một loại nhân tử
maplab [4]	Nhiều cảm biến, nhiều phiên, hiệu chuẩn ngoại trong cùng khung	Kết hợp chương này với C/13 và C/14
OpenVINS [5]	Camera + IMU với hiệu chuẩn ngoại và t_d trực tuyến	Mục 2B — thả ngoại và thời gian ra làm biến

MINI PROJECT SLAM · Thêm odometry bánh xe vào một hệ VIO, rồi đo giá trị thật của nó · 3 ngày

Mục tiêu	đo giá trị của một cảm biến phụ theo cách đúng — không phải bằng ATE trung bình mà bằng tỉ lệ sống sót khi có sự cố.
Dữ liệu	một robot mặt đất có bánh xe encoder, camera và IMU; tự thu ba loại chuỗi: chuỗi sạch, chuỗi có che camera vài giây, và chuỗi có bánh trượt (chạy lên một tấm thảm trơn hoặc nâng một bánh).
Các bước	thêm nhân tử odometry và ràng buộc nonholonomic vào một hệ VIO có sẵn; hiệu chuẩn đường kính bánh và ngoại bánh–camera theo C/14; cài một cổng phát hiện trượt ở mức chuỗi theo mục 2C — so tích lũy odometry với tích lũy IMU trên một cửa sổ; chạy bốn cấu hình (VIO thuần, VIO + odom không cổng, VIO + odom có cổng, VIO + odom + nonholonomic) trên cả ba loại chuỗi.
Xong khi	bạn có một bảng bốn cấu hình nhân ba chuỗi với hai cột: ATE và <i>có sống sót không</i> . Bạn phải quan sát được: (a) trên chuỗi sạch, thêm odometry cải thiện ATE ít — đúng như dự đoán, vì đó không phải giá trị chính của nó; (b) trên chuỗi che camera, chỉ các cấu hình có odometry sống sót; (c) trên chuỗi trượt, cấu hình <i>không cổng</i> tệ hơn cả VIO thuần — xác nhận mục 2C bằng dữ liệu của chính bạn, và đó là kết quả đáng giá nhất của bài này.
Nối với	C/18-danh-gia-va-phuong-phap-luan chuẩn hoá cách đo “tỉ lệ sống sót”; D/19-ben-vung-trong-the-gioi-thuc mở rộng danh mục sự cố cần thử.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Ba điều kiện: (1) *hiệp phương sai trung thực* — trọng số quyết định nguồn này được tin bao nhiêu so với mọi nguồn khác, và khai quá nhỏ làm sai lệch có hệ thống của nó chảy vào toàn bộ nghiệm mà không có gì cảnh báo; (2) *hệ quy chiếu và thời gian đúng* — ngoại và t_d , cả hai nên thả ra làm biến theo C/14; (3) *cổng loại outlier riêng cho từng nguồn*. Điều kiện (3) hay bị bỏ nhất, vì nó đòi hiểu cơ chế hỏng của từng cảm biến chứ không suy ra được từ một công thức chung.
- A2. Vì *bản chất sai số bù nhau*, không phải vì độ chính xác. Sai số thị giác tích lũy — nhỏ ở ngắn hạn, lớn vô hạn ở dài hạn. Sai số GNSS bị chặn — vài mét ở mọi thời điểm, hôm nay cũng như một năm sau. Ghép lại: thị giác cho độ chính xác cục bộ, GNSS cho việc không trôi. Nguyên tắc chung đáng nhớ: khi ghép cảm biến, tìm nguồn có *đặc tính sai số bù nhau*, không phải nguồn

chính xác nhất — hai cảm biến cùng trôi theo cùng kiểu không giúp được nhau. Lưu ý: thêm GNSS đòi ước lượng một phép biến đổi bốn bậc tự do giữa hệ cục bộ và hệ toàn cục, và nó cần quỹ đạo đủ cong để quan sát được yaw.

- A3. Vì chế độ hỏng của hai cảm biến *bù nhau* chứ không trùng nhau. IMU trôi nhưng không bao giờ nói dối — nó luôn đo đúng cái nó đo, chỉ có độ lệch, nên ta *mô hình hoá* độ lệch và ước lượng nó. Odometry không trôi khi bình thường, nhưng khi bánh trượt thì nó nói dối một cách tự tin — báo đã đi ba mét trong khi robot đứng yên quay bánh — và không có mô hình nào cho việc trượt, nên phải *phát hiện và loại*. Hai kiểu sai đòi hai cách xử lý ngược nhau. Kết luận: dùng cả hai, và chính việc chúng kiểm chéo lẫn nhau là cơ chế phát hiện trượt.
- A4. Vì mỗi nguồn có *phân bố outlier riêng*. Ví dụ cụ thể: outlier thị giác là khớp sai — rải rác, độc lập, phần dư rất lớn — nên công Mahalanobis trên từng phép đo bất tốt. Outlier odometry là trượt — *kéo dài cả một đoạn, tương quan* (mọi phép đo sai cùng chiều), và phần dư của từng phép đo riêng lẻ có thể *nhỏ*. Công Mahalanobis không bắt được vì không phép đo nào lệch đủ xa, nhưng tổng của chúng thì lệch rất xa. Cần một công ở *mức chuỗi*: so tích lũy odometry với tích lũy từ nguồn độc lập trên một cửa sổ. Bài học: công phải thiết kế theo cơ chế hỏng của từng nguồn; n loại cảm biến cần n cơ chế phát hiện.
- A5. Đặt nó thành một bài toán *phá gauge*: liệt kê các bậc tự do không quan sát được của cấu hình thuần thị giác (bảy với mono, theo A/05), rồi hỏi mỗi cảm biến phá được bậc nào và với điều kiện gì — Bảng 1.1 là bảng trả lời. Thêm hai tiêu chí nữa để bài toán đầy đủ: (a) *đặc tính sai số có bù nhau không* (tích lũy so với bị chặn); (b) *chế độ hỏng có độc lập không* — hai cảm biến cùng chết khi mất điện hoặc cùng chết trong sương mù thì không cho khả năng phục hồi. Với robot mặt đất, kết quả thường là stereo + IMU + odometry bánh xe, và D/21 lập luận vì sao đó là điểm ngọt.

NÓI LẠI THẬT ĐƠN GIẢN

Hình dung một cuộc họp để đoán giá một ngôi nhà. Mỗi người đưa ra một con số kèm mức tự tin. Cách hợp lý để tổng hợp là trung bình có trọng số theo mức tự tin. Đó chính xác là điều một hệ hợp nhất cảm biến làm.

Từ hình ảnh đó suy ra được cả ba chỗ có thể hỏng.

Nếu một người nói “tôi rất chắc” trong khi thực ra chỉ đoán bừa, ý kiến người đó lấn át cả phòng — và tệ hơn, cuộc họp sẽ kết luận rằng những người kia đang sai chứ không phải người này đang nói quá. Đây là chuyện xảy ra khi bạn khai mức tin cậy của một cảm biến quá cao.

Nếu một người tính bằng đồng và người khác tính bằng đô la, trung bình hoá là vô nghĩa. Với cảm biến, điều này nghĩa là phải biết chúng đặt ở đâu so với nhau và đo vào lúc nào.

Và nếu có người nói nhầm, phải loại — nhưng mỗi người nói nhầm theo một kiểu. Một người thỉnh thoảng nhầm một chữ số, dễ thấy ngay. Một người khác thì nói rất hợp lý nhưng về một ngôi nhà hoàn toàn khác, và cả buổi không ai phát hiện. Hai kiểu ấy cần hai cách bắt. Đây là chỗ mà một quy tắc chung “loại ai lệch quá xa” sẽ bắt kiểu thứ nhất và bỏ sót kiểu thứ hai — đúng cái xảy ra với bánh xe bị trượt.

Nhưng điều quan trọng nhất thì không nằm ở việc trung bình. Giá trị lớn nhất của việc có nhiều người trong phòng không phải là con số chính xác hơn khi mọi người đều tỉnh táo. Đó là lúc *một người ngất xỉu* — cuộc họp vẫn tiếp tục được.

Với một cái máy, “ngất xỉu” nghĩa là camera bị che, đèn tắt, hay đi vào một hành lang trắng trơn. Một hệ chỉ có camera sẽ dừng lại ở đó. Một hệ có thêm vài nguồn khác thì vẫn đi tiếp được — kém chính xác hơn, nhưng vẫn đi. Và với một sản phẩm, khác biệt giữa “kém chính xác” và “dừng lại” là khác biệt giữa được và không được.

Chỗ không rút gọn được: chỗ không rút gọn được là việc *biết mỗi cảm biến nói dối theo kiểu nào*. Không có quy tắc chung; phải hiểu vật lý của từng cảm biến. Bánh xe nói dối khi trượt; GPS nói dối khi tín hiệu dội vào toà nhà; camera nói dối khi có một chiếc xe tải đi ngang chiếm hết khung hình. Ba kiểu nói dối ấy chẳng liên quan gì tới nhau, và mỗi kiểu cần một cách bắt riêng.

Một câu: Thêm một cảm biến là thêm một ý kiến vào cuộc họp — và điều quan trọng nhất không phải nó nói gì, mà là nó có thành thật về mức nó chắc chắn hay không.

I 1.5 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *thêm cảm biến là thêm nhân tử, không đổi bộ giải* — là hệ quả cấu trúc của đồ thị nhân tử (cửa a). Nó bị bác bỏ nếu có một loại phép đo không viết được thành một nhân tử với Jacobian và hiệp phương sai — và có: các phép đo có *cấu trúc rời rạc*, chẳng hạn “robot đang ở trong phòng A” từ một bộ phân loại, không phải một phần dư liên tục. Chúng cần các kỹ thuật khác (ước lượng đa giả thuyết), và đó là một giới hạn thật của khung này chứ không phải một ngoại lệ nhỏ.

Mệnh đề thứ hai — *mỗi nguồn cần cổng riêng* — kiểm được bằng mini project phần chuỗi trượt. Nó bị bác bỏ nếu một cổng chung đủ tốt cho mọi nguồn trên một tập sự cố đa dạng. Tôi cho là không, dựa trên lập luận về phân bố outlier, nhưng mức độ nghiêm trọng thì tôi chưa đo cho các cặp cảm biến ngoài odometry–thị giác.

Mệnh đề thứ ba — *giá trị chính của cảm biến phụ là khả năng phục hồi, không phải độ chính xác* — là một phán đoán về ưu tiên, và nó phụ thuộc ứng dụng. Với một hệ quét đo đặc chạy trong điều kiện kiểm soát, độ chính xác đúng là tiêu chí chính và mệnh đề này sai. Nó đúng cho robot hoạt động không giám sát, tức cho mục tiêu đã nêu ở 00-map (*tôi suy ra từ mục tiêu ấy, chưa đổi chiều nguồn*).

I 1.6 Kết nối

Nói lên trên. A/04-uoc-luong-map-va-bundle-adjustment cho khung mà nguyên tắc ở mục 2 là hệ quả trực tiếp. A/05-observability-gauge-va-suy-bien cho ngôn ngữ để đọc Bảng 1.1: mỗi cảm biến phá một gauge. A/06-bat-dinh-va-lan-truyen-sai-so là chương mà điều kiện một ở mục 2A phụ thuộc hoàn toàn vào — và nó cũng cảnh báo rằng hiệp phương sai đáng tin là thứ khó có. C/14-hieu-chuan-nhu-mot-nhanh-cua-slam là điều kiện hai.

Nói xuống dưới. C/18-danh-gia-va-phuong-phap-luan là chỗ khối *Cần nhớ* trở thành phương pháp: đo một hệ hợp nhất bằng ATE trung bình là bỏ sót phần lớn giá trị của nó. D/19-ben-vung-trong-the-gioi-thuc mở rộng danh mục sự cố mà cảm biến phụ giúp vượt qua. D/21-tu-prototype-len-mass-production là chỗ việc chọn cấu hình cảm biến trở thành một quyết định có chi phí BOM, có ngân sách điện, và có hệ quả về dung sai cơ khí — và là chỗ lập luận “stereo + IMU + odometry là điểm ngọt cho robot mặt đất” được trình bày đầy đủ.

Nói ngang. . . /IMU_Fusion/ là cây đầy đủ về nhánh quán tính: tiền tích phân (chương 7), bộ lọc và tối ưu (chương 8, 9), khởi tạo (chương 10), ràng buộc chuyển động và đa cảm biến (chương 12). Chương này cố ý không lặp lại. . . /marker/ là cây về nhánh fiducial, nơi marker đóng vai trò neo tuyệt đối — và nó cho thấy cái giá thật của giải pháp ấy: lắp đặt, hiệu chuẩn, bảo trì.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu nguyên tắc chung để thêm một cảm biến, và ba điều kiện của nó. Điều kiện nào hay bị bỏ nhất và vì sao?
2. Vì sao một nhân tử có hiệp phương sai sai lại tệ hơn không có nhân tử đó, và vì sao không có gì cảnh báo?
3. Outlier thị giác và outlier odometry khác nhau thế nào về phân bố, và vì sao một cổng chung không bắt được cả hai?
4. Odometry bánh xe và IMU đều cho tỉ lệ metric. Chế độ hỏng của chúng khác nhau thế nào, và vì sao điều đó nghĩa là nên dùng cả hai?

5. Thêm GNSS đòi ước lượng một phép biến đổi mấy bậc tự do, và điều kiện nào để nó quan sát được?
6. Nêu nguyên tắc chọn cảm biến theo “đặc tính sai số bù nhau”, và cho một ví dụ về hai cảm biến *không* bù nhau.
7. Vì sao đo một hệ hợp nhất bằng ATE trung bình trên chuỗi sạch là bỏ sót phần lớn giá trị của nó?

Nguồn

- [1] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [2] F. Dellaert and M. Kaess, “Factor Graphs for Robot Perception,” *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017, mức tin A.
- [3] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [4] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [5] P. Geneva, K. Eickenhoff, W. Lee, et al., “OpenVINS: A Research Platform for Visual-Inertial Estimation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.